

Ecole Nationale des Sciences Appliquees d'AI Hoceima
Universite Abdelmalek Essaadi

Compte Rendu de Travaux Pratiques
Module : Java EE (JEE)

Application de Gestion des Produits

Spring MVC + IOC + JSP + JSTL

Réalisé par : Nabila ES-SAIDY

Encadrant : M. CHERRADI
Transformation Digitale et Intelligence Artificielle (TDIA)
Annee : 2024 - 2025

ENSA AI Hoceima — Filiere Genie Informatique

Introduction

Ce document presente la realisation d'une application web de gestion de produits developpee dans le cadre du module Java EE (JEE). L'application s'appuie sur le framework Spring MVC et respecte l'architecture 3-tiers combinee au patron MVC (Model-View-Controller).

L'objectif principal est de mettre en pratique les concepts fondamentaux suivants :

- Spring MVC : gestion des requetes HTTP via le DispatcherServlet
- Spring IOC : injection de dependances par annotations et par XML
- JSP (Java Server Pages) : generation de pages HTML dynamiques
- JSTL (Java Standard Tag Library) : balises standard pour les vues JSP
- Apache Tomcat : serveur d'application Java EE
- Maven : gestion des dependances et du build

L'application permet de realiser les operations CRUD completes (Create, Read, Update, Delete) sur une liste de produits, avec une interface intuitive et une architecture modulaire et maintenable.

1. Description du Projet

1.1 Objectif

Developper une application web dynamique permettant de gerer une liste de produits a l'aide du framework Spring MVC. L'application implemente les quatre operations fondamentales :

- Afficher la liste des produits
- Ajouter un nouveau produit
- Modifier un produit existant
- Supprimer un produit
- Rechercher un produit par son identifiant

1.2 Donnees Manipulees

Chaque produit est represente par les attributs suivants :

Attribut	Type	Description
idProduit	Long	Identifiant unique du produit (auto-incremente)
nom	String	Nom commercial du produit
description	String	Description detaillee du produit
prix	Double	Prix du produit en dirhams

1.3 Technologies Utilisees

Technologie	Role
Spring MVC 5.3.x	Framework web — gestion des requetes HTTP et navigation
Spring IOC	Conteneur d'injection de dependances (annotations + XML)
JSP (Java Server Pages)	Moteur de templates — generation des vues HTML
JSTL 1.2	Bibliotheque de balises standard pour les JSP
Apache Tomcat 9	Serveur d'application Java EE
Maven	Gestion des dependances et compilation du projet
Java 8+	Langage de programmation principal

2. Architecture du Projet

2.1 Architecture 3-Tiers + MVC

Le projet adopte une architecture 3-tiers qui separe clairement les responsabilites en trois couches independantes, combinee au patron MVC :

Couche	Composant MVC	Role
Presentation	Vue (JSP)	Affichage des donnees — fichiers .jsp avec JSTL
Metier / Logique	Controleur + Modele	Traitement des requetes HTTP et logique applicative
Acces aux donnees	Modele (DAO)	Acces et manipulation des donnees (en memoire)

Le flux de traitement d'une requete suit le chemin suivant :

Flux Spring MVC

1. Le navigateur envoie une requete HTTP (ex: /index.aspx)
2. Le DispatcherServlet intercepte toutes les requetes (*.aspx)
3. Il identifie le Controller correspondant via @RequestMapping
4. Le Controller appelle la couche Service (metier)
5. Le Service appelle la couche DAO pour acceder aux donnees
6. Le Controller place les donnees dans le Model
7. Le ViewResolver resout le nom de la vue en chemin JSP
8. La JSP genere le HTML retourne au navigateur

2.2 Structure du Projet Maven

Le projet est organise selon la structure standard Maven pour un projet web Java EE :

```
GestionDesProduits/
├── pom.xml                                <- Dependances Maven
├── src/main/java/
│   ├── dao/
│   │   ├── Produit.java                  <- Classe modele (entite)
│   │   ├── ProduitDAO.java              <- Interface DAO
│   │   └── ProduitImpl.java              <- Implementation DAO (en memoire)
│   ├── services/
│   │   ├── ProduitMetier.java            <- Interface Service metier
│   │   └── ProduitImplMetier.java        <- Implementation Service
│   └── controller/
│       └── ProduitController.java        <- Controleur Spring MVC
└── src/main/webapp/
    ├── index.jsp                         <- Page d'accueil (redirection)
    ├── Pages/
    │   └── produits.jsp                  <- Vue principale (JSP + JSTL)
    └── WEB-INF/
```

```
| web.xml                <- Deploiement et configuration Tomcat
| spring-beans.xml       <- IoC XML : beans DAO & Service
| application-servlet-config.xml <- IoC Annotations + ViewResolver
```

2.3 Separation des Contextes Spring

L'architecture distingue deux contextes Spring, chacun avec une responsabilite precise :

Fichier de configuration	Contexte	Contenu
application-servlet-config.xml	DispatcherServlet (MVC)	Scan @Controller, ViewResolver JSP, mvc:annotation-driven
spring-beans.xml	Racine (optionnel)	Beans DAO et Service en XML

3. Configuration du Projet

3.1 Dependances Maven (pom.xml)

Le fichier pom.xml declare les dependances necessaires au projet. Maven les telecharge automatiquement depuis le repository central :

```
<dependencies>
  <!-- Spring MVC : framework web complet -->
  <dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-webmvc</artifactId>
    <version>5.3.30</version>
  </dependency>

  <!-- JSTL : balises standard pour les JSP -->
  <dependency>
    <groupId>javax.servlet</groupId>
    <artifactId>jstl</artifactId>
    <version>1.2</version>
  </dependency>

  <!-- Servlet API : fournie par Tomcat (scope provided) -->
  <dependency>
    <groupId>javax.servlet</groupId>
    <artifactId>javax.servlet-api</artifactId>
    <version>4.0.1</version>
    <scope>provided</scope>
  </dependency>
</dependencies>
```

3.2 Fichier web.xml

Le descripteur de deployment configure le comportement global de l'application dans Tomcat. Il declare le DispatcherServlet et le mapping des URL :

```
<!-- Declaration du DispatcherServlet (point d'entree Spring MVC) -->
<servlet>
  <servlet-name>action</servlet-name>
  <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-
class>
  <init-param>
    <param-name>contextConfigLocation</param-name>
    <param-value>/WEB-INF/application-servlet-config.xml</param-value>
  </init-param>
  <load-on-startup>1</load-on-startup>
</servlet>

<!-- Toutes les URLs *.aspx sont traitees par Spring MVC -->
<servlet-mapping>
  <servlet-name>action</servlet-name>
  <url-pattern>*.aspx</url-pattern>
</servlet-mapping>
```

Remarque importante

Un seul DispatcherServlet est declare dans web.xml. Le ContextLoaderListener n'est pas utilise afin d'eviter un double demarrage du contexte Spring, qui causerait des problemes d'injection de dependances entre les couches.

3.3 Fichier application-servlet-config.xml

Ce fichier configure le contexte Spring lie au DispatcherServlet. Il regroupe en un seul endroit la configuration MVC (annotations) et les beans metier (XML) :

```
<!-- Activation des annotations Spring MVC (@Controller, @RequestMapping...) -->
<mvc:annotation-driven/>

<!-- Scan automatique du package 'controller' pour detecter @Controller -->
<context:component-scan base-package="controller"/>

<!-- ViewResolver : mappe le nom de vue retourne par le Controller en JSP -->
<!-- Exemple : retourner "produits" => affiche /Pages/produits.jsp -->
<bean class="org.springframework.web.servlet.view.InternalResourceViewResolver">
  <property name="prefix" value="/Pages/" />
  <property name="suffix" value=".jsp" />
</bean>

<!-- Bean DAO (injection par XML) -->
<bean id="objDAO" class="dao.ProduitImpl" init-method="init"/>

<!-- Bean Service avec injection du DAO -->
<bean id="objServices" class="services.ProduitImplMetier">
  <property name="dao" ref="objDAO"/>
</bean>
```

4. Principe de l'Inversion de Controle (IoC)

4.1 Concept

L'Inversion de Controle (IoC) est un principe fondamental de Spring. Au lieu que le développeur crée et gère les objets manuellement, c'est le conteneur Spring qui prend en charge leur cycle de vie.

Approche	Code	Avantage
Sans IoC (couplage fort)	<code>new ProduitImplMetier()</code>	Aucun — dépendance directe
Avec IoC (couplage faible)	<code>@Autowired ProduitMetier service</code>	Modulaire, testable, remplaçable

4.2 Configuration par Annotations

Le Controller utilise les annotations Spring pour être détecté et injecté automatiquement :

```
@Controller // Declare ce bean comme un Controller
Spring
public class ProduitController {

    @Autowired // Spring injecte automatiquement le bean
    ProduitMetier services; // correspondant à l'interface
    ProduitMetier

    @RequestMapping(value="/index")
    public String pageIndex(Model model) {
        model.addAttribute("listeProduit", services.getAllProduits());
        return "produits"; // -> ViewResolver -> /Pages/produits.jsp
    }
}
```

4.3 Configuration par XML

Les couches DAO et Service sont configurées via XML dans `application-servlet-config.xml`. Le bean DAO est injecté dans le bean Service via la propriété 'dao' :

```
<!-- ProduitImpl est instancié par Spring et sa méthode init() est appelée -->
<bean id="objDAO" class="dao.ProduitImpl" init-method="init"/>

<!-- ProduitImplMetier reçoit le bean objDAO via le setter setDao() -->
<bean id="objServices" class="services.ProduitImplMetier">
    <property name="dao" ref="objDAO"/>
</bean>
```

Dans la classe `ProduitImplMetier`, le setter correspondant permet à Spring d'injecter la dépendance :

```
public class ProduitImplMetier implements ProduitMetier {
    private ProduitDAO dao;

    // Spring appelle ce setter pour injecter le bean objDAO
    public void setDao(ProduitDAO dao) {
        this.dao = dao;
    }
}
```

5. Implementation Couche par Couche

5.1 Couche Modele — dao/Produit.java

La classe Produit est le modele de donnees central de l'application. Elle contient les attributs d'un produit et les getters/setters necessaires a Spring MVC pour le data binding automatique :

```
package dao;

public class Produit {
    private Long idProduit;    // Identifiant unique
    private String nom;        // Nom du produit
    private String description; // Description
    private Double prix;       // Prix

    public Produit() {}        // Constructeur vide requis par Spring MVC

    public Produit(String nom, String description, Double prix) {
        this.nom = nom;
        this.description = description;
        this.prix = prix;
    }
    // Getters et setters...
}
```

5.2 Couche DAO

Interface ProduitDAO

L'interface definit le contrat CRUD sans preciser l'implementation. Cela permet de changer la source de donnees (memoire, base de donnees, etc.) sans modifier les couches superieures :

```
package dao;
import java.util.List;

public interface ProduitDAO {
    void addProduit(Produit p);
    void deleteProduit(Long id);
    Produit getProduitById(Long id);
    List<Produit> getAllProduits();
    void updateProduit(Produit p);
}
```

Implementation ProduitImpl

L'implementation utilise une ArrayList en memoire pour simuler une base de donnees. La methode init() est appelee par Spring au demarrage (init-method="init") pour pre-charger des produits de test :

```
public class ProduitImpl implements ProduitDAO {
    private List<Produit> produits = new ArrayList<>();
}
```

```
// Appele automatiquement par Spring au demarrage
public void init() {
    System.out.println("Spring IOC est bien fonctionnee !");
    addProduit(new Produit("PC 1", "Sony Vaio 1", 7000.0));
    addProduit(new Produit("PC 2", "Sony Vaio 2", 6000.0));
    // ... autres produits de test
}

@Override
public void addProduit(Produit p) {
    p.setIdProduit((long) (produits.size() + 1));
    produits.add(p);
}

@Override
public void updateProduit(Produit p) {
    for (Produit existing : produits) {
        if (existing.getIdProduit().equals(p.getIdProduit())) {
            existing.setNom(p.getNom());
            existing.setDescription(p.getDescription());
            existing.setPrix(p.getPrix());
            break;
        }
    }
}
}
```

5.3 Couche Service

La couche service joue le role d'intermediaire entre le Controller et le DAO. Elle encapsule la logique metier et peut etre etendue avec des regles de validation, de calcul, etc. :

```
public class ProduitImplMetier implements ProduitMetier {
    private ProduitDAO dao;

    // Setter pour l'injection Spring (XML : <property name="dao"
    ref="objDAO"/>)
    public void setDao(ProduitDAO dao) { this.dao = dao; }

    @Override public void addProduit(Produit p)        { dao.addProduit(p); }
    @Override public void deleteProduit(Long id)        {
        dao.deleteProduit(id); }
    @Override public List<Produit> getAllProduits()    { return
        dao.getAllProduits(); }
    @Override public Produit getProduitById(Long id)   { return
        dao.getProduitById(id); }
    @Override public void updateProduit(Produit p)     { dao.updateProduit(p); }
}
}
```

5.4 Couche Web — Controller

Le ProduitController est le composant central de Spring MVC. Il intercepte les requetes HTTP, appelle le service metier, et retourne le nom de la vue a afficher :

```
@Controller
public class ProduitController {

    @Autowired
    ProduitMetier services;

    // Afficher la liste complete des produits
    @RequestMapping(value="/index")
    public String pageIndex(Model model) {
        model.addAttribute("listeProduit", services.getAllProduits());
        return "produits";
    }

    // Rechercher un produit par ID
    @RequestMapping(value="/searchProduct")
    public String searchProduct(Model model, @RequestParam Long idProduit) {
        List<Produit> liste = new ArrayList<>();
        Produit p = services.getProduitById(idProduit);
        if (p != null) liste.add(p);
        model.addAttribute("listeProduit", liste);
        return "produits";
    }

    // Ajouter un produit (Spring fait le data binding automatiquement)
    @RequestMapping(value="/addProduct")
    public String addProduct(Model model, Produit p) {
        if (p.getNom() != null && !p.getNom().trim().isEmpty())
            services.addProduit(p);
        model.addAttribute("listeProduit", services.getAllProduits());
        return "produits";
    }

    // Charger un produit pour modification (GET)
    @RequestMapping(value="/editProduit", method=RequestMethod.GET)
    public String editProduit(Model model, @RequestParam Long id) {
        model.addAttribute("produitEdit", services.getProduitById(id));
        model.addAttribute("listeProduit", services.getAllProduits());
        return "produits";
    }

    // Mettre a jour un produit (POST)
    @RequestMapping(value="/updateProduit", method=RequestMethod.POST)
    public String updateProduit(Model model, @RequestParam Long idProduit,
        @RequestParam String nom, @RequestParam String description,
        @RequestParam Double prix) {
        Produit p = new Produit(); p.setIdProduit(idProduit);
        p.setNom(nom); p.setDescription(description); p.setPrix(prix);
        services.updateProduit(p);
        model.addAttribute("listeProduit", services.getAllProduits());
        return "produits";
    }

    // Supprimer un produit
    @RequestMapping(value="/deleteProduit")
    public String supprimerProduit(Model model, @RequestParam Long id) {
```

```
        services.deleteProduit(id);  
        model.addAttribute("listeProduit", services.getAllProduits());  
        return "produits";  
    }  
}
```

6. Vue JSP — produits.jsp

6.1 Description de la Vue

Le fichier produits.jsp est la vue principale de l'application. Il utilise la bibliothèque JSTL (Java Standard Tag Library) pour afficher dynamiquement les données transmises par le Controller via le Model.

La vue remplit trois fonctions :

- Formulaire de recherche par ID
- Formulaire dynamique d'ajout / modification (selon la présence de produitEdit dans le Model)
- Tableau affichant la liste des produits avec les liens Modifier et Supprimer

6.2 Formulaire Dynamique Ajout / Modification

Le même formulaire gère à la fois l'ajout et la modification grâce aux expressions EL (Expression Language). Si le Model contient un attribut 'produitEdit', le formulaire se passe en mode modification :

```
<!-- Action dynamique : updateProduit si edition, addProduct si ajout -->
<form action="${produitEdit != null ? 'updateProduit' : 'addProduct'}.aspx"
      method="post">

    <!-- ID cache : renseigne uniquement en mode modification -->
    <input type="hidden" name="idProduit"
          value="${produitEdit != null ? produitEdit.idProduit : ''}" />

    <!-- Champs pre-remplis en mode modification, vides en mode ajout -->
    <input type="text" name="nom"
          value="${produitEdit != null ? produitEdit.nom : ''}" />

    <!-- Bouton dynamique -->
    <input type="submit"
          value="${produitEdit != null ? 'Mettre a jour' : 'Ajouter'}" />

</form>
```

6.3 Boucle JSTL forEach

La balise JSTL <c:forEach> itère sur la liste transmise par le Controller (model.addAttribute("listeProduit", ...)) et génère une ligne de tableau pour chaque produit :

```
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>

<table border="1">

<tr><th>ID</th><th>NOM</th><th>DESCRIPTION</th><th>PRIX</th><th>Options</th></tr>

    <c:forEach items="${listeProduit}" var="o">
```

```
<tr>
  <td>${o.idProduit}</td>
  <td>${o.nom}</td>
  <td>${o.description}</td>
  <td>${o.prix}</td>
  <!-- Lien de suppression (requete GET avec parametre id) -->
  <td><a href="deleteProduit.aspx?id=${o.idProduit}">supprimer</a></td>
  <!-- Lien de modification (charge le produit dans le formulaire) -->
  <td><a href="editProduit.aspx?id=${o.idProduit}">Modifier</a></td>
</tr>
</c:forEach>
</table>
```

7. URLs et Fonctionnalites

7.1 Tableau des URLs

Le DispatcherServlet intercepte toutes les URL se terminant par .aspx et les route vers le Controller appropriée :

URL	Methode	Description
/GestionDesProduits/	GET	Page d'accueil (redirection vers index.aspx)
/GestionDesProduits/index.aspx	GET	Affiche la liste complete des produits
/GestionDesProduits/searchProduct.aspx	POST	Recherche un produit par son ID
/GestionDesProduits/addProduct.aspx	POST	Ajoute un nouveau produit
/GestionDesProduits/editProduit.aspx?id=X	GET	Pre-remplit le formulaire pour modifier le produit X
/GestionDesProduits/updateProduit.aspx	POST	Met a jour le produit apres modification
/GestionDesProduits/deleteProduit.aspx?id=X	GET	Supprime le produit dont l'ID est X

7.2 Annotations Spring MVC Utilisees

Annotation	Localisation	Role
@Controller	ProduitController	Declare la classe comme Controller Spring MVC
@Autowired	ProduitController	Injection automatique du bean ProduitMetier
@RequestMapping	Methodes du Controller	Mappe une URL HTTP vers une methode Java
@RequestParam	Parametres des methodes	Recupere un parametre de la requete HTTP

9. Conclusion

Ce projet illustre concrètement la mise en oeuvre de l'architecture Spring MVC combinée à l'injection de dépendances (IoC). Les principaux apports sont :

- Séparation claire des responsabilités : Controller, Service, DAO sont indépendants
- Inversion de contrôle : Spring gère le cycle de vie des beans, pas le développeur
- Configuration hybride : XML pour DAO/Service, Annotations pour le Controller
- Réutilisabilité : les interfaces permettent de changer l'implémentation sans toucher au reste
- Maintenabilité : ajouter une fonctionnalité ne nécessite de modifier qu'une couche

Ce travail constitue une base solide pour aborder des projets Java EE plus avancés, notamment avec JPA/Hibernate pour la persistance en base de données réelle, ou Spring Boot pour simplifier la configuration.